

FIT LADDER • SPLIT WHAT CAN'T PAGINATE

The read- across carousel

A comparison can't be cut down the
middle — so when it won't fit, it
becomes a sequence.

What the carousel solves

- A list overflows? Split it between items — that is **auto-split**
- A **comparison** reads *across* its two sides — slice it and the meaning is gone
- So the engine **re-authors** it as a sequence: a cover, one page per side, a verdict
- Each frame stands alone, reads in order — no setting, no shrinking

Scoring model: before and after the calibration loop

Side by side →

SCORING MODEL: BEFORE AND AFTER THE
CALIBRATION LOOP

Before Calibration

Equal weights —
confidence, recency,
relevance at 33%
each. Honest about
the guessing.

Unchanged in
eighteen months.

SCORING MODEL: BEFORE AND AFTER THE
CALIBRATION LOOP

After Calibration

Weights reflect each team's accuracy — but the team keeps changing, and the history is one atypical quarter.

**The evaluation came
down to a question
the vendors could not
answer**

Side by side →

Buy a vendor framework

Three evaluated.

None expose
calibration weights —
the criterion that
eliminated all three,
added after the team
chose to build.

THE EVALUATION CAME DOWN TO A QUESTION
THE VENDORS COULD NOT ANSWER

Build the framework in-house

Owens the policy, the
loop, the timeline.

The window closes in
eighteen months; a
cutover eats nine.

THE VERDICT

The conclusion came first;
the evaluation was built to
hold it. Everyone in the room
knew it.

Decisions used to require a quarterly re-litigation

Side by side →

DECISIONS USED TO REQUIRE A QUARTERLY RE-LITIGATION

Before

Decisions lived in the room they were made in. Six months on, nobody could say why a project was killed.

DECISIONS USED TO REQUIRE A QUARTERLY RELITIGATION

After

Every decision is logged with its signals, options, and bet. Teams still relitigate, but the argument is shorter.

SECTION 01 • DEEP DIVE

Scoring Model Deep Dive

The most configurable component. Six dimensions:

The supporting detail →

Confidence

Corroborating sources, 1–5.

Enterprise counts as 1.

Recency

Time-decay, configurable half-life. Set to two weeks.

Strategic Relevance

Owner-scored, 1–5. A 5 tracks whoever presents the roadmap.

Source Diversity

Rewards breadth — in practice, newsletter volume.

Volatility

Penalizes swing — including the signal that caught the last surprise.

Owner Bias

The correction nobody applies, scored by the owner it is meant to correct.

The six signal dimensions, what they measure, and how they score

Entry by entry →

Confidence

Independent corroborating
sources 1–5, enterprise counts
as one

Recency

Time-decay on a configurable
half-life Two-week default
surprises everyone

Strategic Relevance

Owner-scored against the
roadmap 1–5, and a 5 is

whoever presents

Source Diversity

Rewards breadth of
corroboration Counts
newsletters as sources

Volatility

Penalizes signals that swing
Also penalizes the early-
warning ones

Owner Bias

The correction nobody applies
Scored by the owner it corrects

DECISION • 2026 Q1

**We are
building, not
buying**

The reasoning →

WE ARE BUILDING, NOT BUYING

Build

Owns the scoring policy, the calibration loop, and the timeline — plus the maintenance, the on-call, and every future explanation of why the framework scored the wrong thing.

WE ARE BUILDING, NOT BUYING

Why not buy

Three vendors, none exposing calibration weights. The weights are the product; you cannot buy the product without them. That was the finding.

WE ARE BUILDING, NOT BUYING

Why not delay

The window closes in eighteen months — a sentence that has been in this deck, unchanged, since Q1 2025.

BEFORE & AFTER • SCORING MECHANICS

Spreadsheet-driven scoring versus a framework that is basically also a spreadsheet

Both implementations →

BEFORE · THE HONEST SPREADSHEET

```
import pandas as pd

signals = pd.read_csv("signals.csv")
signals["score"] = signals.apply(
    lambda r: 0.33 * r.confidence +
    0.33 * r.recency + 0.33 *
    r.relevance,
    axis=1,
)
signals.to_csv("scored.csv")
```

AFTER • THE FRAMEWORK

```
from decision_framework import
Calibrator

cal =
Calibrator.load("weights.json")
signals["score"] =
cal.score(signals)
cal.decisions.log_if_changed(signals
)
```

One finish, more slides

- **compare-prose / split-panel / list-tabular** → a cover, then the content flows on
- **decision** → the verdict is the cover; its reasons flow on
- **compare-code** → a cover, then one code block per page
- Every split wears the *same* cover finish — the deck, just more of it (`autosplit: on`)

More slides, never a clipped comparison

*One cover finish
across the family
— nothing shrinks*